

OS LAB

Skill-4

1. Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution

CODE:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_TOKENS 100
#define MAX_TOKEN_LEN 100

typedef enum {
    WORD,
    PIPE,
    INPUT_REDIRECT,
    OUTPUT_REDIRECT,
    APPEND,
    BACKGROUND,
    SEMICOLON
} TokenType;

typedef struct {
    char value[MAX_TOKEN_LEN];
    TokenType type;
} Token;

/* Convert token type to readable text */
const char *typeName(TokenType type)
{
    switch (type) {
        case WORD:
            return "WORD";
        case PIPE:
```

```

        return "PIPE";
    case INPUT_REDIRECT:
        return "INPUT_REDIRECT";
    case OUTPUT_REDIRECT:
        return "OUTPUT_REDIRECT";
    case APPEND:
        return "APPEND";
    case BACKGROUND:
        return "BACKGROUND";
    case SEMICOLON:
        return "SEMICOLON";
    default:
        return "UNKNOWN";
    }
}

/* Add a token to the token array */
void addToken(Token tokens[], int *count, const char *value, TokenType type)
{
    if (*count >= MAX_TOKENS) {
        printf("Error: Too many tokens\n");
        return;
    }

    strcpy(tokens[*count].value, value);
    tokens[*count].type = type;
    (*count)++;
}

/* Tokenization */
int tokenize(char *input, Token tokens[])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0') {

        /* Ignore whitespace */
        if (input[i] == ' ' ||
            input[i] == '\t' ||
            input[i] == '\n') {
            i++;
            continue;
        }
    }
}

```

```

/* Pipe */
if (input[i] == '|') {
    addToken(tokens, &count, "|", PIPE);
    i++;
    continue;
}

/* Input redirection */
if (input[i] == '<') {
    addToken(tokens, &count, "<", INPUT_REDIRECT);
    i++;
    continue;
}

/* Output redirection */
if (input[i] == '>') {
    if (input[i + 1] == '>') {
        addToken(tokens, &count, ">>", APPEND);
        i += 2;
    } else {
        addToken(tokens, &count, ">", OUTPUT_REDIRECT);
        i++;
    }

    continue;
}

/* Background */
if (input[i] == '&') {
    addToken(tokens, &count, "&", BACKGROUND);
    i++;
    continue;
}

/* Semicolon */
if (input[i] == ';') {
    addToken(tokens, &count, ";", SEMICOLON);
    i++;
    continue;
}

/* Normal word */

```

```

char word[MAX_TOKEN_LEN];
int j = 0;

while (input[i] != '\0' &&
      input[i] != ' ' &&
      input[i] != '\t' &&
      input[i] != '\n' &&
      input[i] != '|' &&
      input[i] != '<' &&
      input[i] != '>' &&
      input[i] != '&' &&
      input[i] != ';') {

    if (j < MAX_TOKEN_LEN - 1) {
        word[j++] = input[i];
    }

    i++;
}

word[j] = '\0';

if (j > 0) {
    addToken(tokens, &count, word, WORD);
}
}

return count;
}

/* Validate token sequence */
int validateTokens(Token tokens[], int count)
{
    if (count == 0) {
        printf("Empty command.\n");
        return 0;
    }

    /* Pipe cannot be first */
    if (tokens[0].type == PIPE) {
        printf("Syntax Error: Pipe cannot be first.\n");
        return 0;
    }
}

```

```

/* Pipe cannot be last */
if (tokens[count - 1].type == PIPE) {
    printf("Syntax Error: Pipe cannot be last.\n");
    return 0;
}

for (int i = 0; i < count - 1; i++) {

    /* Two consecutive pipes */
    if (tokens[i].type == PIPE &&
        tokens[i + 1].type == PIPE) {

        printf("Syntax Error: Consecutive pipes.\n");
        return 0;
    }

    /* Redirection must have a filename */
    if ((tokens[i].type == INPUT_REDIRECT ||
        tokens[i].type == OUTPUT_REDIRECT ||
        tokens[i].type == APPEND) &&
        tokens[i + 1].type != WORD) {

        printf("Syntax Error: Redirection requires a filename.\n");
        return 0;
    }
}

return 1;
}

/* Display tokens */
void printTokens(Token tokens[], int count)
{
    printf("\n--- TOKENS ---\n");

    for (int i = 0; i < count; i++) {
        printf("[%d] %s -> %s\n",
            i,
            tokens[i].value,
            typeName(tokens[i].type));
    }
}

/* Simple parser */

```

```

void parse(Token tokens[], int count)
{
    printf("\n--- PARSE RESULT ---\n");

    printf("Command structure:\n");

    for (int i = 0; i < count; i++) {

        if (tokens[i].type == PIPE) {
            printf("    |\n");
            printf("    PIPE\n");
        }
        else if (tokens[i].type == WORD) {
            printf("    WORD: %s\n", tokens[i].value);
        }
        else {
            printf("    %s: %s\n",
                typeName(tokens[i].type),
                tokens[i].value);
        }
    }
}

```

```

int main()
{
    char input[1000];

    Token tokens[MAX_TOKENS];

    printf("Mini Shell Parser\n");
    printf("-----\n");

    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 1;
    }

    int count = tokenize(input, tokens);

    printTokens(tokens, count);

    if (!validateTokens(tokens, count)) {
        return 1;
    }
}

```

```

}

parse(tokens, count);

printf("\nParsing completed successfully.\n");

return 0;
}

```

OUTPUT:

```

lahari@DESKTOP-AIC33F7:~$ cd OSLab
lahari@DESKTOP-AIC33F7:~/OSLab$ mkdir skill4
lahari@DESKTOP-AIC33F7:~/OSLab$ cd skill4
lahari@DESKTOP-AIC33F7:~/OSLab/skill4$ ls -l | grep ".c"
lahari@DESKTOP-AIC33F7:~/OSLab/skill4$ mkdir parser_project
lahari@DESKTOP-AIC33F7:~/OSLab/skill4$ cd parser_project
lahari@DESKTOP-AIC33F7:~/OSLab/skill4/parser_project$ nano parser.c
lahari@DESKTOP-AIC33F7:~/OSLab/skill4/parser_project$ gcc parser.c -o parser
lahari@DESKTOP-AIC33F7:~/OSLab/skill4/parser_project$ ./parser
Mini Shell Parser
-----
Enter command: ls -l | grep ".c"

--- TOKENS ---
[0] ls -> WORD
[1] -l -> WORD
[2] | -> PIPE
[3] grep -> WORD
[4] ".c" -> WORD

--- PARSE RESULT ---
Command structure:
  WORD: ls
  WORD: -l
  |
  PIPE
  WORD: grep
  WORD: ".c"

Parsing completed successfully.

```

```
Parsing completed successfully.
lahari@DESKTOP-AIC33F7:~/OSLab/skill4/parser_project$ ./parser
Mini Shell Parser
-----
Enter command: ls | grep.c

--- TOKENS ---
[0] ls -> WORD
[1] | -> PIPE
[2] grep.c -> WORD

--- PARSE RESULT ---
Command structure:
  WORD: ls
    |
  PIPE
  WORD: grep.c

Parsing completed successfully.
```

```
Parsing completed successfully.
lahari@DESKTOP-AIC33F7:~/OSLab/skill4/parser_project$ ls > output.txt
lahari@DESKTOP-AIC33F7:~/OSLab/skill4/parser_project$ ./parser
Mini Shell Parser
-----
Enter command: ls > output.txt

--- TOKENS ---
[0] ls -> WORD
[1] > -> OUTPUT_REDIRECT
[2] output.txt -> WORD

--- PARSE RESULT ---
Command structure:
  WORD: ls
  OUTPUT_REDIRECT: >
  WORD: output.txt

Parsing completed successfully.
```


Parsing completed successfully.

lahari@DESKTOP-AIC33F7:~/OSLab/skill4/parser_project\$./parser

Mini Shell Parser

Enter command: ls >> output.txt

--- TOKENS ---

[0] ls -> WORD

[1] >> -> APPEND

[2] output.txt -> WORD

--- PARSE RESULT ---

Command structure:

WORD: ls

APPEND: >>

WORD: output.txt

Parsing completed successfully.

lahari@DESKTOP-AIC33F7:~/OSLab/skill4/parser_project\$ |